Cache and Memory Performance Profiling

Ethan Katz

Table of Contents

Introduction

Zero-Queue Baselines

- Latency vs Working-Set Size
- Latency by Working-Set Category

Pattern and Granularity Sweep

- Effective Throughput
- Time per Updated Element

Read/Write Mix Sweep

- Throughput by Mix
- Time per Operation with Dependent Reads

Intensity Sweep

- Throughput vs Latency
- Knee and Little's Law

Working-Set Sweep

- Latency Transition Regions
- Bandwidth Across Working Sets

Cache-Miss Impact

- Runtime vs Far-Access Fraction
- Inferred Cost per Far Access

TLB-Miss Impact

- Latency vs Page Locality
- Runtime vs Page Configuration

Benchmark Pitfall

- Cache Reuse vs a Large Working Set
- Pinned vs Unpinned Execution

Conclusion

Honest AI Usage

Introduction

Modern CPUs put L1, L2 and L3 caches in front of DRAM to hide how slow memory is. Each level has its own latency, bandwidth and size, so the same loop can run much faster or slower depending on where its data sits and how many requests it has in flight. This report measures that hierarchy on one laptop and explains the results in terms of locality, concurrency, cache misses and TLB misses.

The test system is a ThinkPad X1 Carbon Gen 13 with an Intel Core Ultra 7 255U, running Ubuntu 24.04 (Linux 6.17) with g++ 13.3 (-O2 -march=native). It's a hybrid chip. The benchmarks could land on CPUs 0 to 3, which are two P-cores with two hyperthreads each (48 KiB L1D, 2 MiB L2, 12 MiB shared L3, 64-byte lines), or on CPUs 12 and 13, which are low-power E-cores with 32 KiB L1D, 2 MiB L2 and no L3. Unless I say otherwise, every new measurement is pinned to CPU 0 with taskset. I left the governor at powersave and ran on battery, so CPU 0 sat at about 2.9 GHz under load instead of its 5.2 GHz maximum. Every configuration was repeated five times (twelve for the pinning study), and plots show the median with interquartile-range whiskers.

I couldn't use hardware performance counters. The installed perf doesn't match the running kernel, and perf_event_paranoid is set to 4, which blocks perf_event_open for normal users. A quick probe showed that hardware, cache, TLB and even software events all return "Permission denied". So the cache-miss and TLB-miss sections rely on timing, kernel page-fault counts and checked huge-page backing instead of miss counts, and I point that out where it matters.

The report goes through the eight required experiments, each with its own section and two figures: zero-queue baselines, a pattern and stride sweep, a read/write mix sweep, an intensity sweep with Little's Law, a working-set sweep, cache-miss impact, TLB-miss impact, and one benchmark pitfall. A short conclusion wraps it up.

Zero-Queue Baselines

This experiment measures how long one dependent load takes at different working-set sizes. The code walks a random cyclic chain of 64-byte nodes, so every load has to wait for the one before it. Nothing overlaps, and what's left is the zero-queue latency of whichever level is serving the data.

The table converts the pinned re-run (Experiment 5) to cycles at the measured 2.9 GHz.

Each footprint is named after the nominal level it fits in, but that doesn't prove that level actually served the loads.

Footprint	Fits nominal	Original run (ns)	Pinned re-run (ns)	Cycles at 2.9 GHz
16 KiB	L1D (48 KiB)	3.31	3.77	11
512 KiB	L2 (2 MiB)	10.0	11.6	34
3 MiB	L3 (12 MiB)	34.2	30.3	88
64 MiB	DRAM	403.2	404.8	1174

Latency vs Working-Set Size

Figure A shows the original, unpinned run at 14 sizes from 16 KiB to 64 MiB with 10 million dependent loads per trial. Latency stays between 3.3 and 3.8 ns up to 48 KiB, rises to 6.0 ns at 64 KiB, climbs from 10 to 17.6 ns between 512 KiB and 2 MiB, and reaches 27.6 to 34.2 ns at 3 to 4 MiB.

The biggest jump is between 4 and 8 MiB, where latency shoots up to 318.7 ns even though 8 MiB should fit in the 12 MiB L3. Oddly, the 12 MiB point (256.4 ns) is faster than 8 MiB, and the curve levels off at 345 to 403 ns from 16 to 64 MiB. The L1 transition matches the 48 KiB capacity, but the L3 one doesn't.

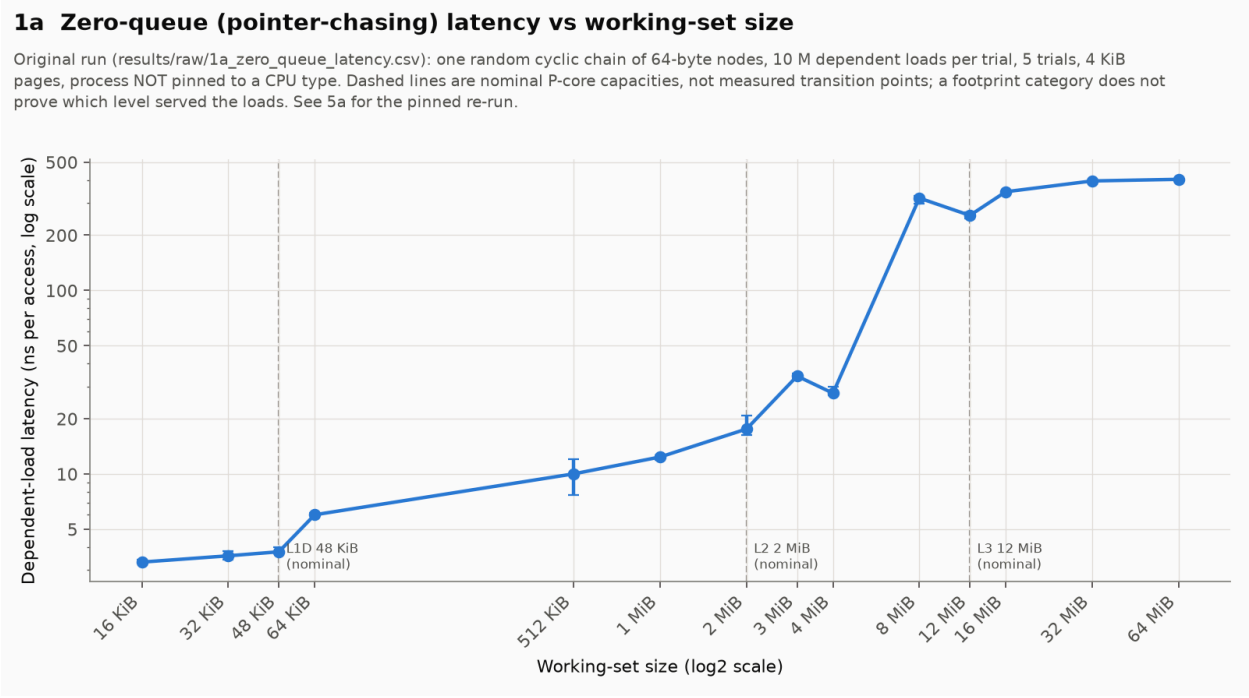


Figure A. Pointer-chasing latency vs. working-set size (original run, 5 trials, log axes). Dashed lines mark nominal P-core capacities.

Latency by Working-Set Category

Figure B groups the same data into footprint categories relative to the nominal capacities. Representative medians are 3.6 ns at or below L1D, 6.0 ns just above it, 12.4 ns at or below L2, 34.2 ns just above L2, 318.7 ns at 8 to 12 MiB and 395.0 ns above L3.

The "at or below L3" category ends up close to DRAM rather than near the 3 to 4 MiB group. Keep in mind these are pointer-chasing latencies for specific footprints, not clean L1, L2, L3 or DRAM latencies. The pinned re-run in Figure I shows the same early rise.

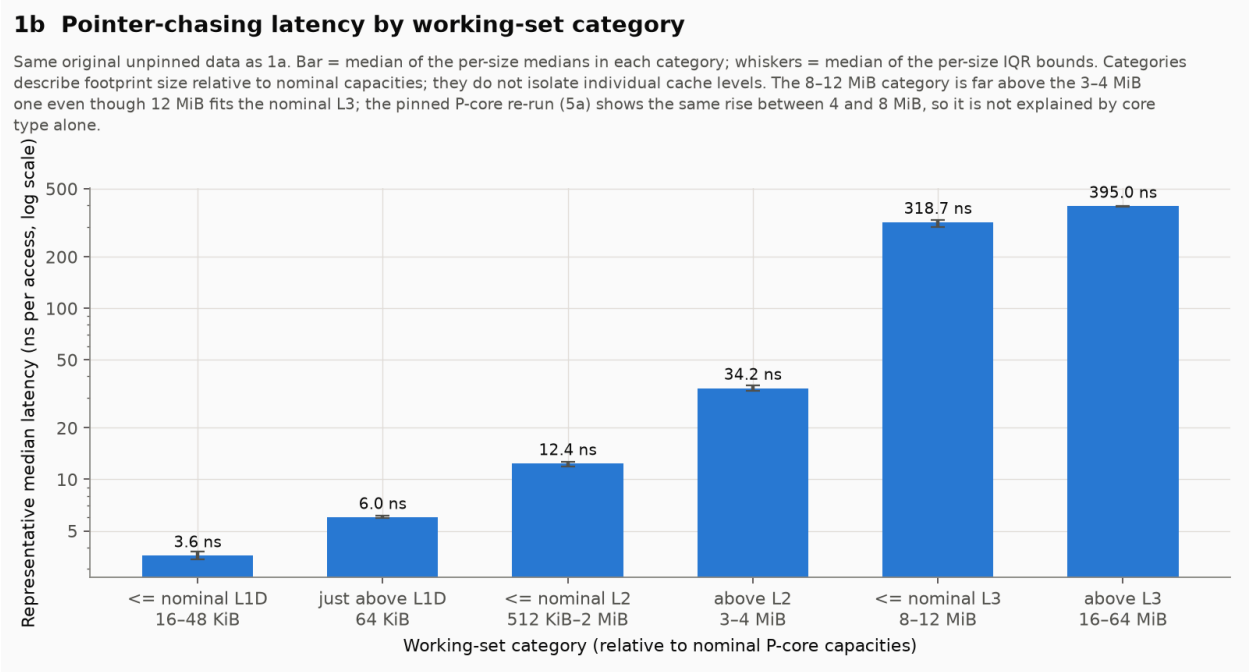


Figure B. Median latency by working-set category (original run).

Pattern and Granularity Sweep

This experiment updates one 8-byte element every 64, 256 or 1024 bytes of a 64 MiB buffer, in sequential or randomized order (data[i] += 1 plus a checksum), for 10 passes and 5 trials. The point is to see how spatial locality and prefetching affect throughput once the data doesn't fit in cache anymore.

Effective Throughput

Figure C reports a software-estimated throughput of 16 bytes (one load and one store) per update. Sequential order reaches 2.489 GB/s at a 64-byte stride, but only 0.468 GB/s at 256 bytes

and 0.383 GB/s at 1024 bytes. Randomized order gives 0.531, 0.392 and 0.409 GB/s.

Order only helps in the sequential 64-byte case. There, back-to-back updates hit back-to-back lines, which the hardware prefetcher can stream in. At 256 bytes and up, every update lands on a new line no matter the order, so both cost about one memory access per update and end up near 0.4 GB/s.

This number isn't DRAM bandwidth. Each update actually moves a full 64-byte line, four times the 16 bytes I counted, and the 8-byte index vector gets streamed in the same timed loop too.

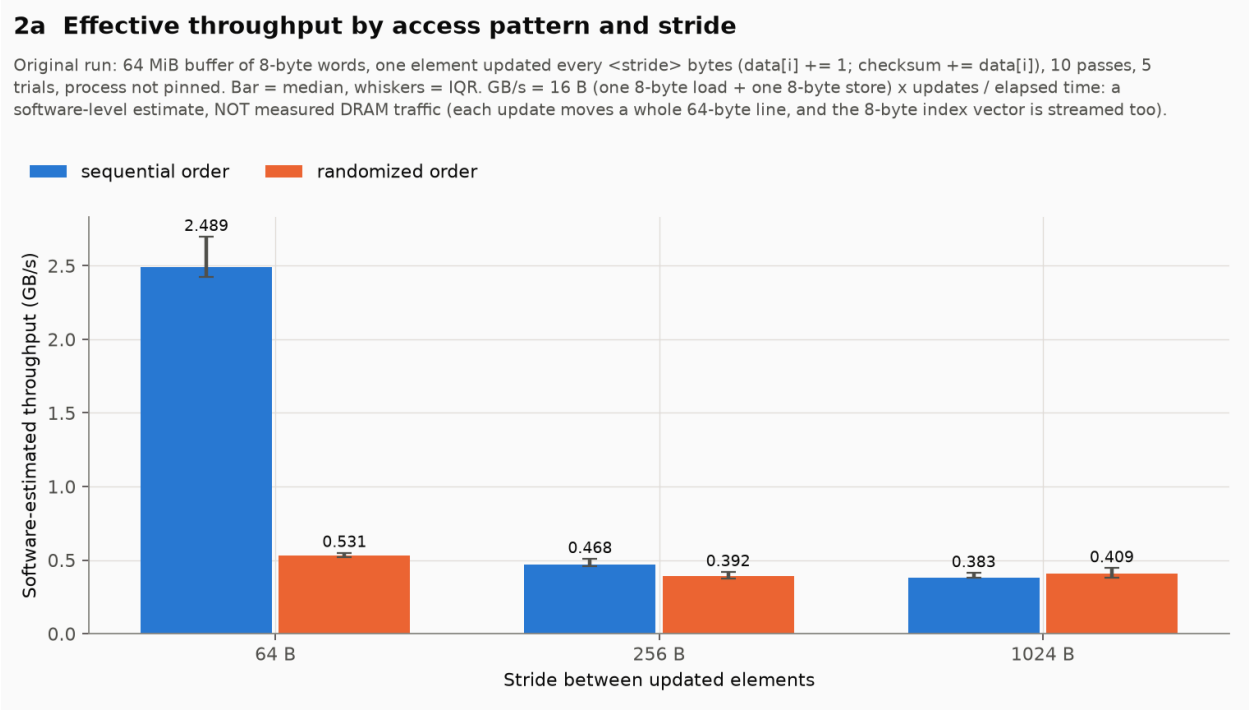


Figure C. Software-estimated throughput by access pattern and stride (64 MiB, 5 trials).

Time per Updated Element

Figure D divides elapsed time by the number of updates, because a 64-byte stride performs 1,048,576 updates per pass while a 1024-byte stride performs only 65,536. Sequential updates take 6.4, 34.2 and 41.8 ns per element at the three strides, and randomized updates take 30.1, 40.8 and 39.2 ns.

That 30 to 42 ns range is way below the 400 ns pointer-chase latency at the same footprint, because these updates don't depend on each other and lots of misses overlap. The stride-1024 runs only last 15 to 28 ms, so they're the most affected by timer resolution and

frequency changes.

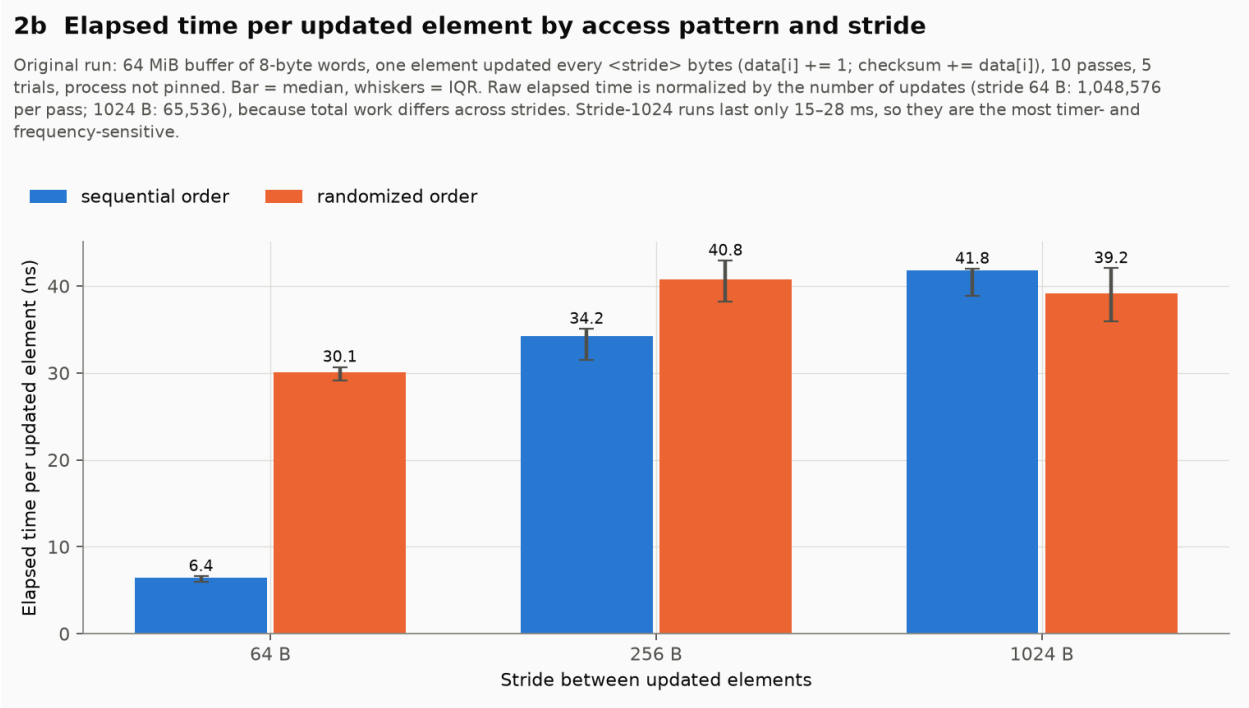


Figure D. Elapsed time per updated element by access pattern and stride.

Read/Write Mix Sweep

This experiment compares 100% reads, 100% writes, 70/30 and 50/50 mixes of random 8-byte operations at two footprints: 1 MiB, which fits the nominal L2, and 256 MiB, which exceeds the L3. Addresses come from an xorshift generator kept in registers, so there's no index array adding traffic. Writes are spread out evenly with a fixed pattern so the read/write branch stays predictable, and each footprint uses the same operation count for every mix.

Throughput by Mix

Figure E shows independent operations, where many requests can overlap. At 1 MiB, throughput is 297, 333, 331 and 360 million operations per second for 100% read, 70/30, 50/50 and 100% write. At 256 MiB it drops to 29.9, 31.0, 31.3 and 55.3 million per second (0.24 to 0.44 GB/s of payload).

Pure writes win at both sizes. A store can retire into the store buffer without waiting for its line, but a load has to wait for its data. The payload number only counts 8 bytes per operation and leaves out the read-for-ownership and write-back each store causes, so it isn't DRAM

bandwidth.

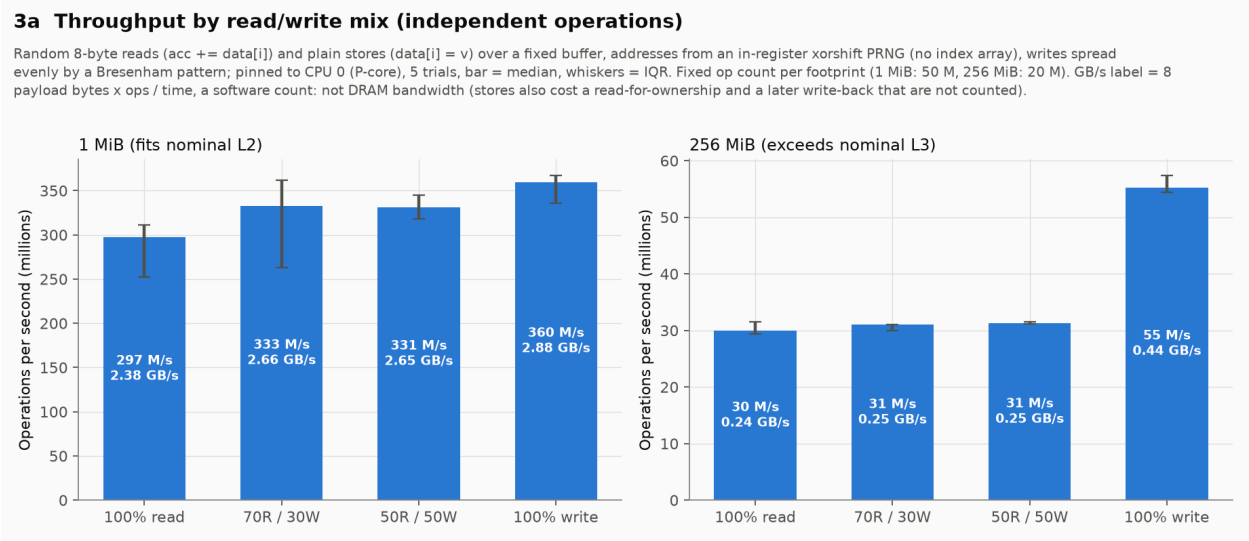


Figure E. Throughput by read/write mix with independent operations (5 trials).

Time per Operation with Dependent Reads

Figure F makes every read depend on the previous one, which exposes read latency. At 256 MiB the mean time per operation is 418.5, 294.8, 214.3 and 19.6 ns for 100% read, 70/30, 50/50 and 100% write. Just blending the two pure endpoints predicts 298.8 and 219.0 ns for the mixed cases, which is within 2.2% of what I measured. So each read pays a fixed DRAM latency, and the writes barely add anything.

At 1 MiB the values are 11.5, 9.2, 6.3 and 3.5 ns, and the 50/50 case comes in 16% faster than the linear prediction. This is just elapsed time divided by total operations, not a separate read or write latency.

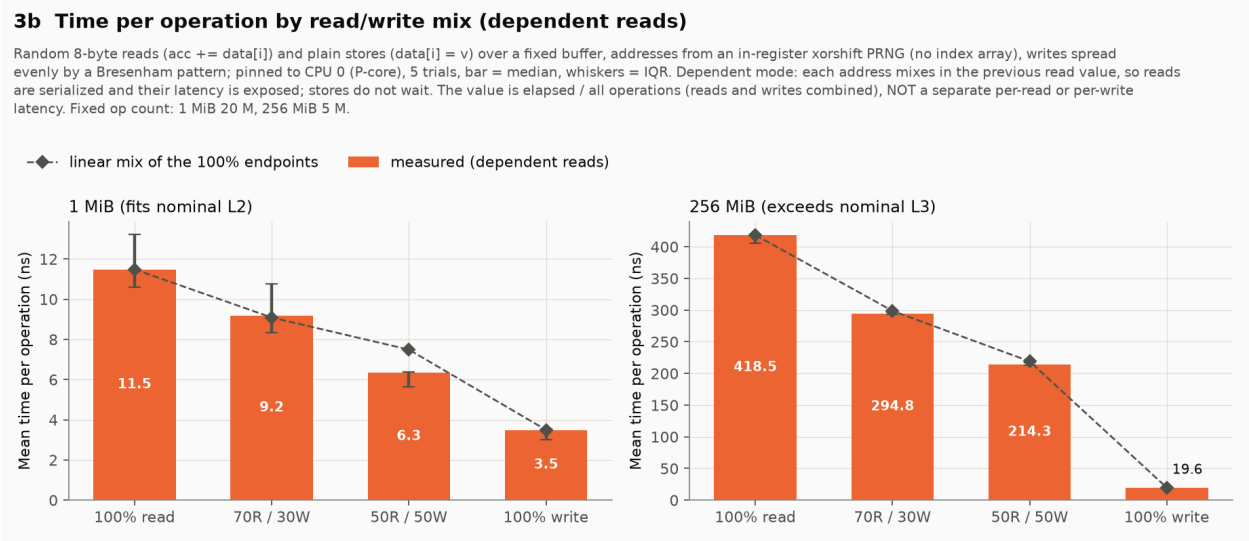


Figure F. Time per operation by mix with dependent reads, compared with a linear mix of the endpoints.

Intensity Sweep

This experiment controls how many memory requests are in flight at once. Each thread walks K independent pointer chains through one 256 MiB list backed by 2 MiB huge pages. A chain can only have one load outstanding, so the concurrency is L = threads × K. Throughput λ is completed loads per second, and response time W is the average time between successive loads of one chain. One thread ran on CPU 0, two on CPUs 0 and 2, and four on CPUs 0 to 3.

Throughput vs Latency

Figure G plots λ against W. With one thread, throughput grows from 2.9 to 64.6 million loads per second as L goes from 1 to 32, while W stays between 350 and 450 ns until it rises to 495 ns at L = 32. Two threads reach 129.7 million per second at L = 64.

With four threads, throughput reaches 186.8 million per second at L = 96 and stays at 185.2 million at L = 128, while W rises from 514 to 691 ns. That's the knee. Past it, more concurrency just adds queueing delay. W also varies about 10% between runs at low L, which is

why the curve zigzags there.

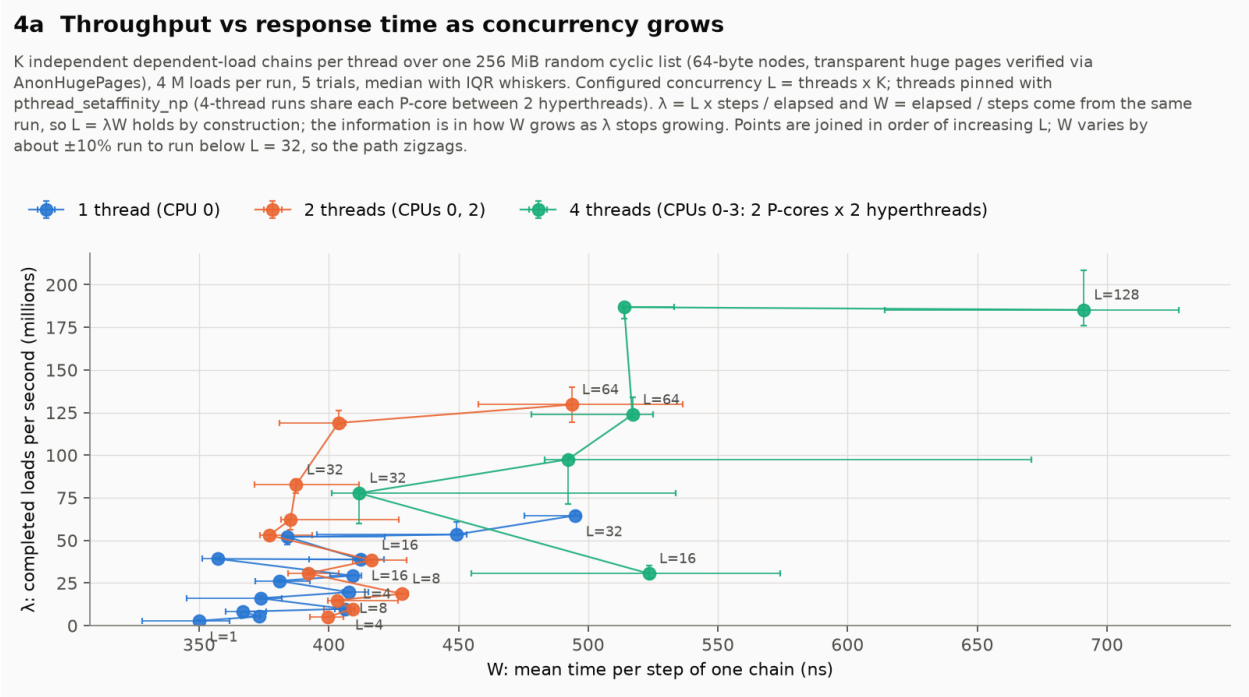


Figure G. Throughput vs. response time as concurrency grows (256 MiB, huge pages, 5 trials).

Knee and Little's Law

Little's Law states L = λW. Since λ and W both come from the same elapsed time, the law holds automatically. At L = 128 with four threads, 185.2 million loads per second times 691 ns gives 128 requests in flight. A more useful number is λ × W0, where W0 = 350 ns is the single-chain response time. It tells you how many loads would need to overlap at unloaded latency to get the measured throughput.

Figure H shows λ × W0 tracking the configured L up to about 32 and then falling below it, reaching 65 at L = 128. The one- and two-thread curves never flatten out (their last steps still gain 21% and 9%), so their knee is past what I tested. Only the four-thread curve levels off, near L = 96 and about 12 GB/s of cache-line traffic. Those runs only last 20 to 30 ms and split each core between two hyperthreads, so the knee location is a rough estimate.

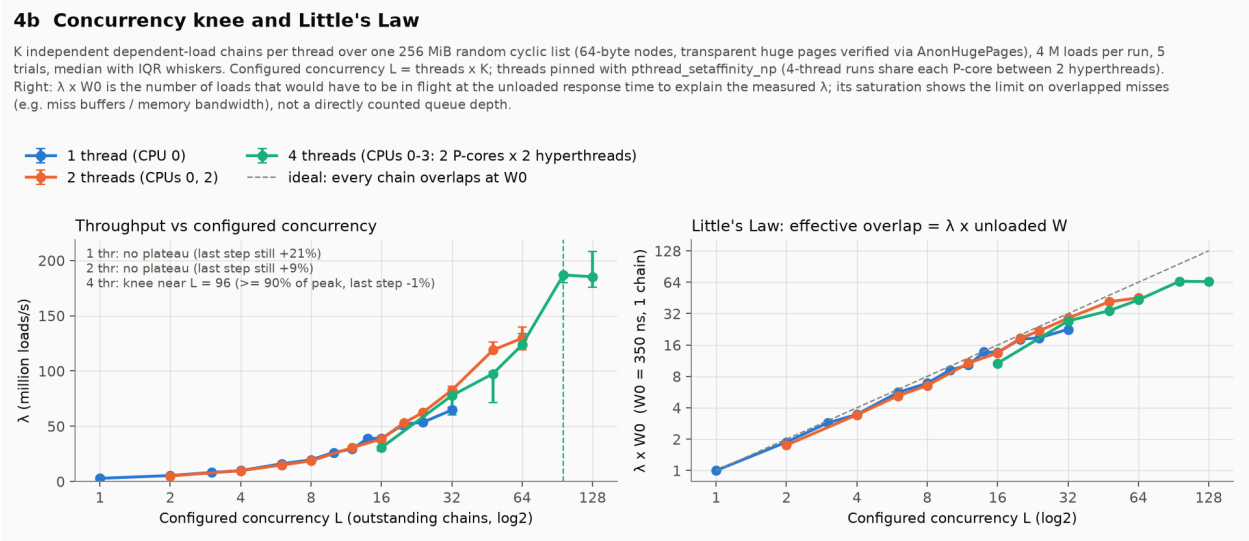


Figure H. Throughput vs. configured concurrency (left) and effective overlap λ × W0 vs. configured concurrency (right).

Working-Set Sweep

This experiment reruns the pointer chase on a finer grid of 34 sizes from 4 KiB to 256 MiB, pinned to a P-core. It also adds a streaming benchmark that reads or writes every word of a buffer, so I can measure bandwidth at each size.

Latency Transition Regions

Figure I compares the pinned sweep with the original run. Latency is 3.4 to 4.4 ns up to 48 KiB and jumps to 9.2 ns at 56 KiB, just past the 48 KiB L1D. It stays at 8.4 to 11.6 ns up to 768 KiB, then rises gradually to 23.6 ns at 2 MiB and 30.3 ns at 3 MiB as the footprint approaches the 2 MiB L2.

The step to near-DRAM latency again happens between 4 MiB (54.3 ns) and 8 MiB (334 ns), well before the nominal 12 MiB L3, and the curve flattens at 357 to 419 ns from 10 MiB to 256 MiB. Since the pinned P-core shows the same early rise as the unpinned run, core type can't be the whole story. Page-translation cost, L3 sharing and power-managed L3 behaviour on battery could all play a part, but I can't tell them apart from these measurements. The large footprints use 4 KiB pages, so they also include page-walk cost (see TLB-Miss Impact).

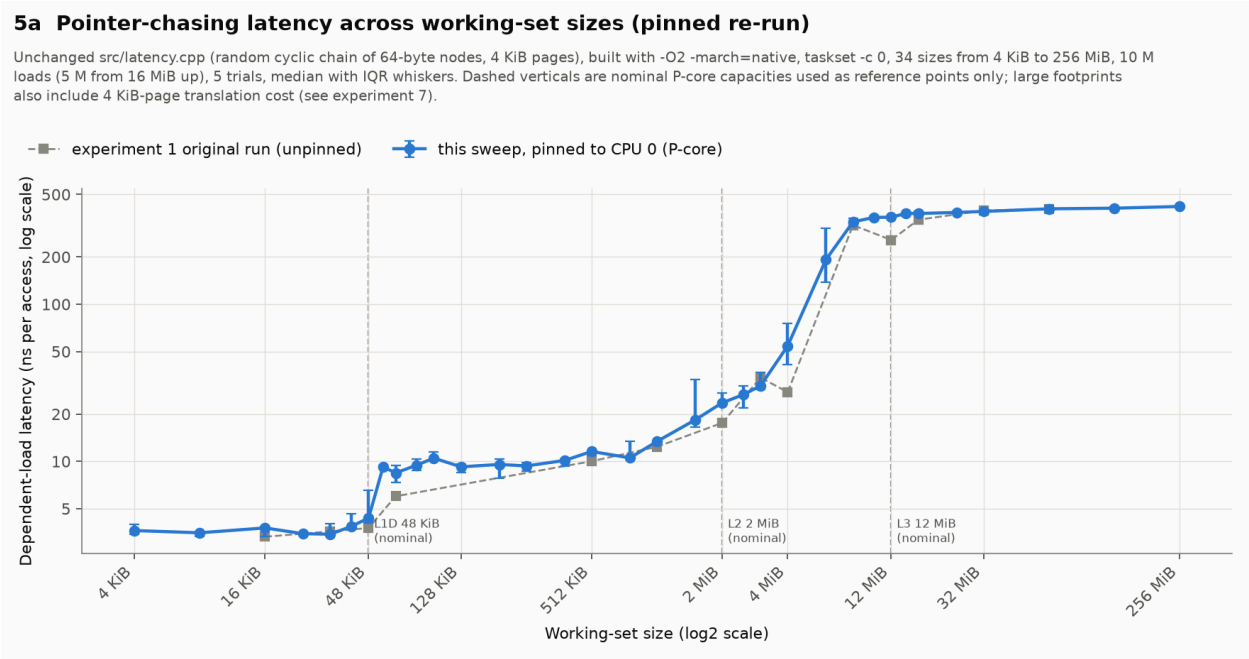


Figure I. Pinned pointer-chasing latency across working-set sizes, with the original run for comparison.

Bandwidth Across Working Sets

Figure J shows single-thread streaming bandwidth from an AVX2-vectorized read loop and a store loop, each moving about 2 GiB per measurement. Reads reach 54 to 60 GB/s up to 128 KiB, 47 to 55 GB/s up to 2 MiB, about 30 GB/s at 4 to 6 MiB, and 11 to 12 GB/s from 16 MiB upward.

Writes reach 73 to 75 GB/s while the buffer fits L1D, drop to about 45 GB/s from 64 KiB to 1 MiB, and settle at 8 to 9 GB/s beyond 32 MiB. The drops happen in the same places as on the latency curve. These are bytes counted in software from one thread, and the write numbers leave out read-for-ownership and write-back, so neither curve is a real measured DRAM bandwidth.

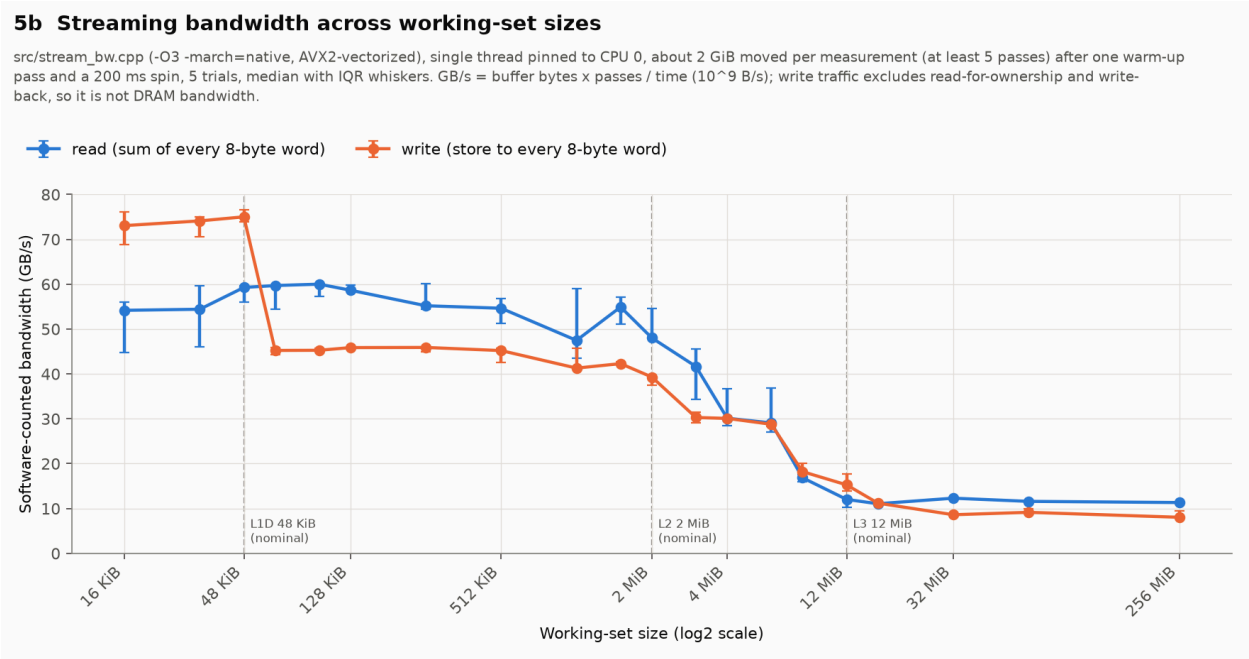


Figure J. Single-thread streaming read and write bandwidth across working-set sizes.

Cache-Miss Impact

Since I didn't have hardware counters, this experiment sets up misses on purpose instead of counting them. One dependent chain sends a designed fraction f of its loads to random lines of a far buffer (1 MiB, 8 MiB or 256 MiB) and the rest to a 16 KiB near buffer that stays in L1. This follows the AMAT form, time per load = T_near + f × (T_far − T_near), so the slope of runtime against f estimates the extra cost of one far access.

Runtime vs Far-Access Fraction

Figure K shows runtime rising in a straight line with f. For the 256 MiB far buffer, time per load grows from about 5 ns at f = 0 to 356 ns at f = 1. For the 8 MiB buffer it reaches about 79 ns with wide spread between trials, and for the 1 MiB buffer it barely moves.

Note that f is chosen by the program, not a measured miss rate, and the 1 MiB far buffer won't necessarily miss L2.

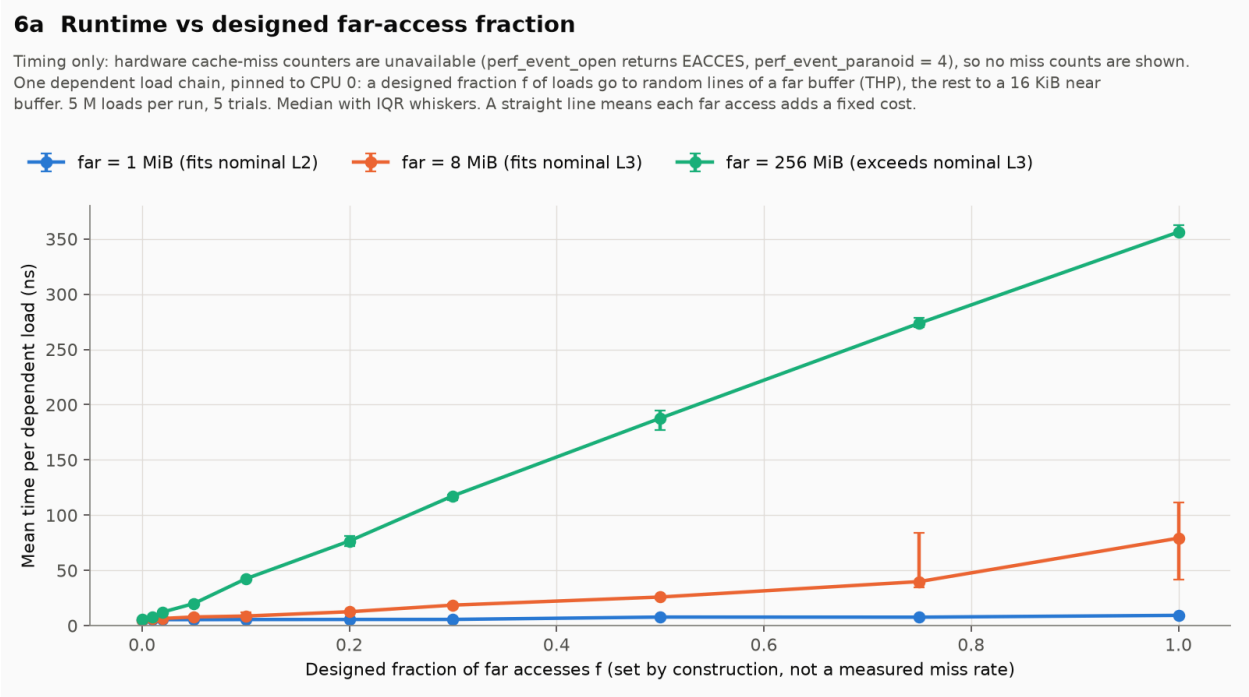


Figure K. Time per dependent load vs. designed far-access fraction for three far-buffer sizes.

Inferred Cost per Far Access

Figure L reports the fitted slope for each far buffer. A far access costs 359.3 ns extra for 256 MiB (minimum R^2 0.997), close to the 362 ns pointer-chase latency with huge pages in the TLB experiment. The 8 MiB buffer gives 81.5 ns with an interquartile range of 53 to 88 ns and a minimum R^2 of 0.61, consistent with some trials hitting in L3 and others going to DRAM. The 1 MiB buffer adds only 3.5 ns.

These are inferred costs, not measured miss penalties or miss counts. I had planned to correlate counted misses with runtime, but couldn't without perf access.

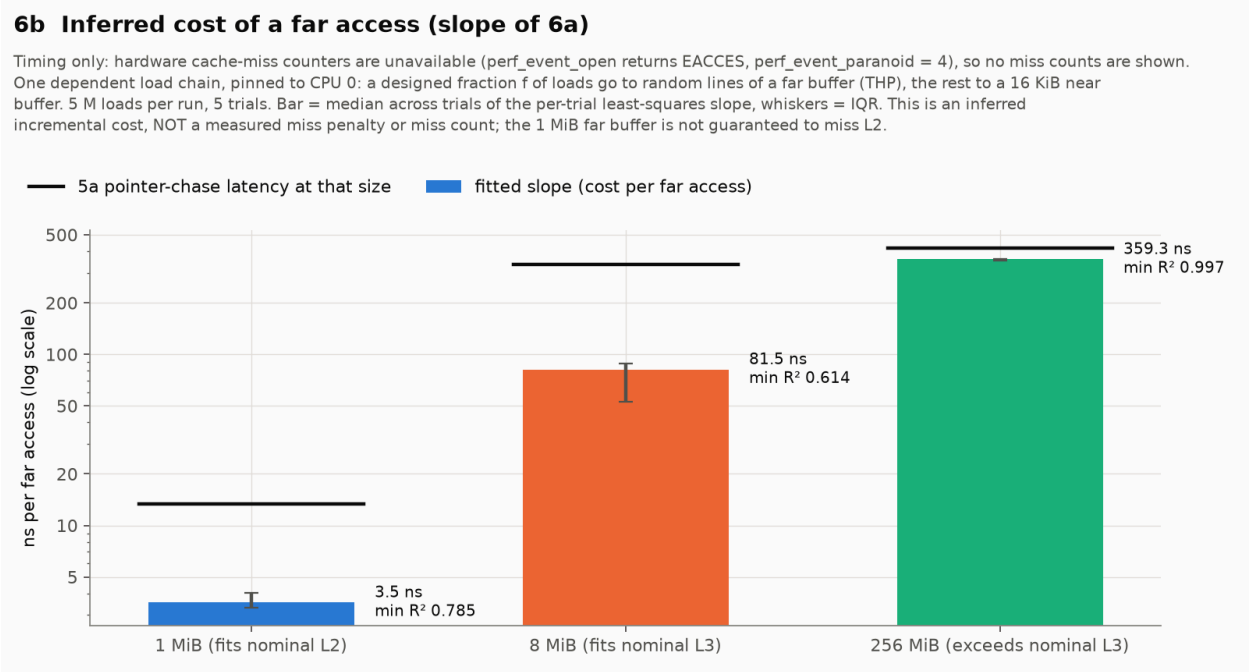


Figure L. Inferred incremental cost of a far access (slope of Figure K), with the pointer-chase latency at the same size for reference.

TLB-Miss Impact

This experiment changes page locality while keeping the data size the same, and compares normal 4 KiB pages with 2 MiB transparent huge pages requested through madvise (no root needed). I didn't have TLB-miss counters, so the evidence comes from time per load, minor page faults during setup and the huge-page backing the kernel reports.

Latency vs Page Locality

Figure M keeps the same 8192 cache lines (512 KiB, smaller than the 2 MiB L2) and spreads them over 128 to 8192 pages. With 4 KiB pages, latency stays at 9 to 10 ns up to 2048 pages, then rises to 15.8 ns at 4096 pages and 21.0 ns at 8192 pages. With huge pages it stays between 5.9 and 9.0 ns throughout.

The only thing changing is the number of pages, so the slowdown is most likely address translation. It shows up once the touched 4 KiB pages go past roughly 2048 pages (8 MiB). That's inferred from timing, though, not from counted TLB misses.

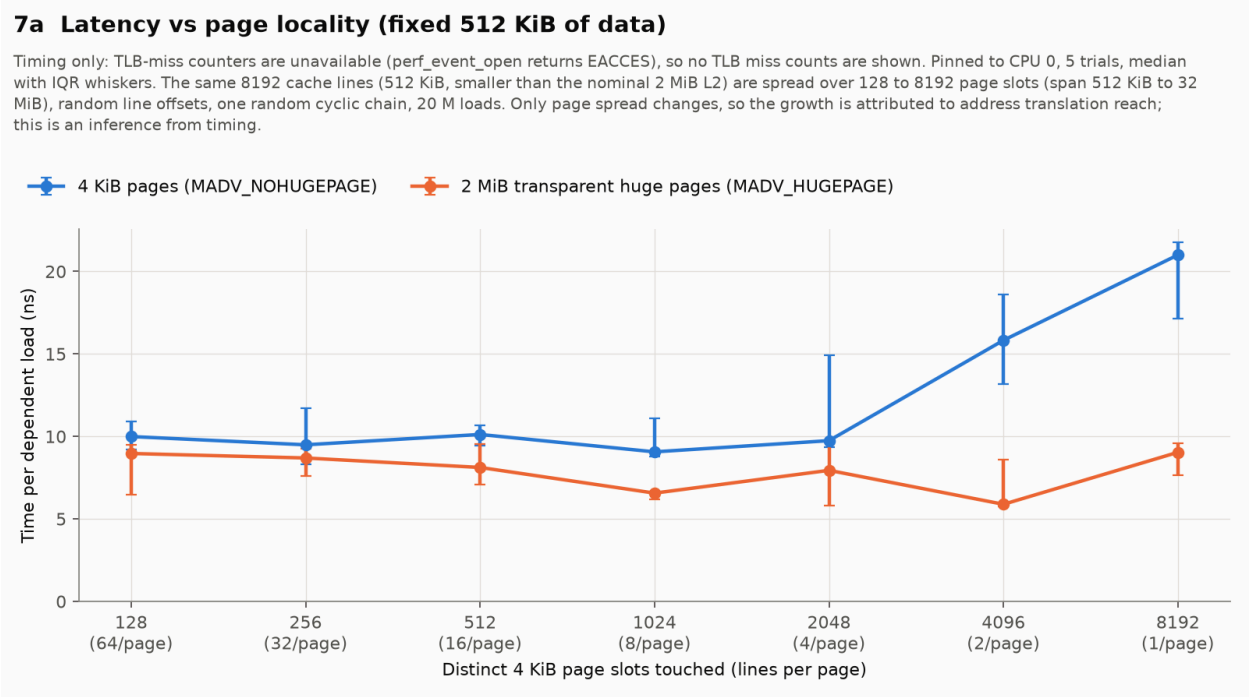


Figure M. Time per load vs. number of pages touched, 4 KiB pages vs. huge pages (fixed 512 KiB of data).

Runtime vs Page Configuration

Figure N compares dense random chases over 2 to 256 MiB. Huge pages are faster at every size: 10.0 vs. 15.4 ns at 2 MiB, 326 vs. 360 ns at 16 MiB, 354 vs. 396 ns at 64 MiB, and 362 vs. 421 ns at 256 MiB, a 10 to 15% gain for the large sets. Setup page faults drop from 65,537 to 129 at 256 MiB, confirming that 2 MiB pages were used.

Huge pages aren't guaranteed, though. In one 256 MiB trial only 44 MiB got huge-page backing, and that run took 405 ns, close to the 4 KiB result. That backs up translation cost as the reason for the gap.

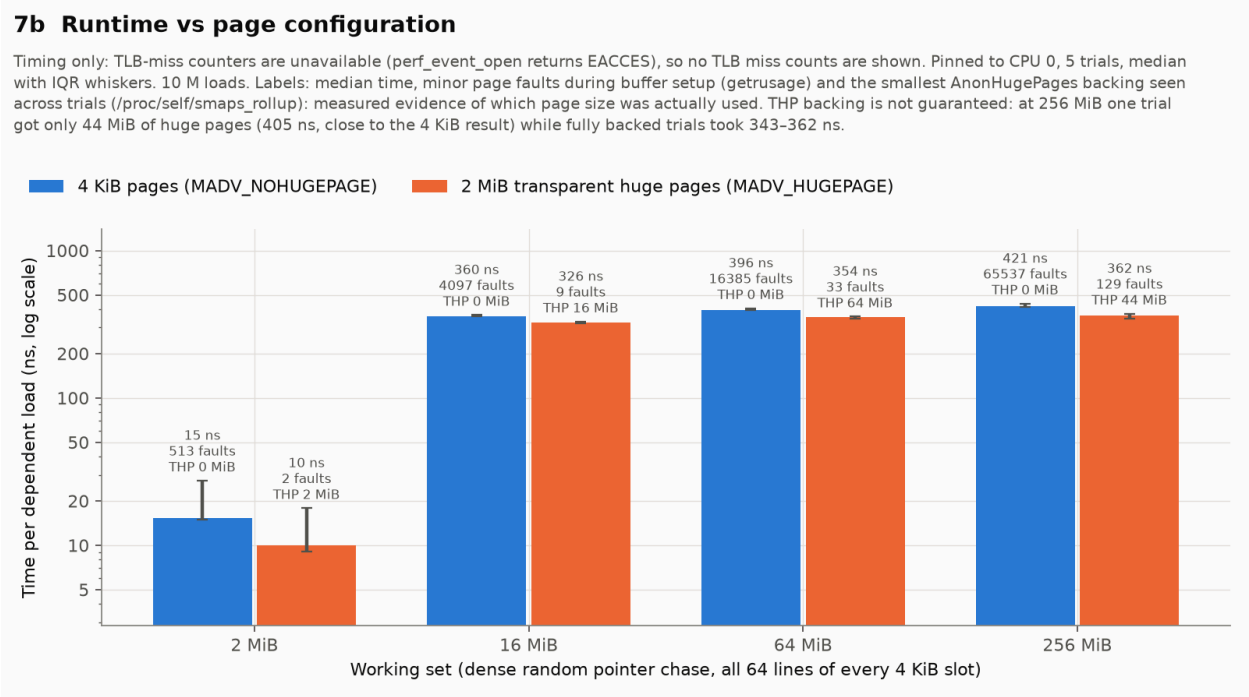


Figure N. Time per load by working set and page size, labelled with measured page faults and huge-page backing.

Benchmark Pitfall

This section shows two easy ways for a memory benchmark to give you the wrong number: accidentally measuring cache reuse instead of memory, and letting the scheduler pick the core on a hybrid CPU.

Cache Reuse vs a Large Working Set

Figure O times each of 20 read passes separately after evicting the caches with a 64 MiB read. A 1 MiB buffer reads at 9.6 GB/s on the first, cold pass and reaches 39.2 GB/s by pass 10, 4.1 times faster. A 256 MiB buffer cannot be reused from cache and stays between 9.4 and 12.9 GB/s on every pass.

If you reported the warm 1 MiB number as memory bandwidth, you'd overstate it several times over. The warm passes aren't even stable. The second pass is still at cold speed, and later passes drift back down to about 24 GB/s. I didn't pin down why, but power management on battery is one possibility.

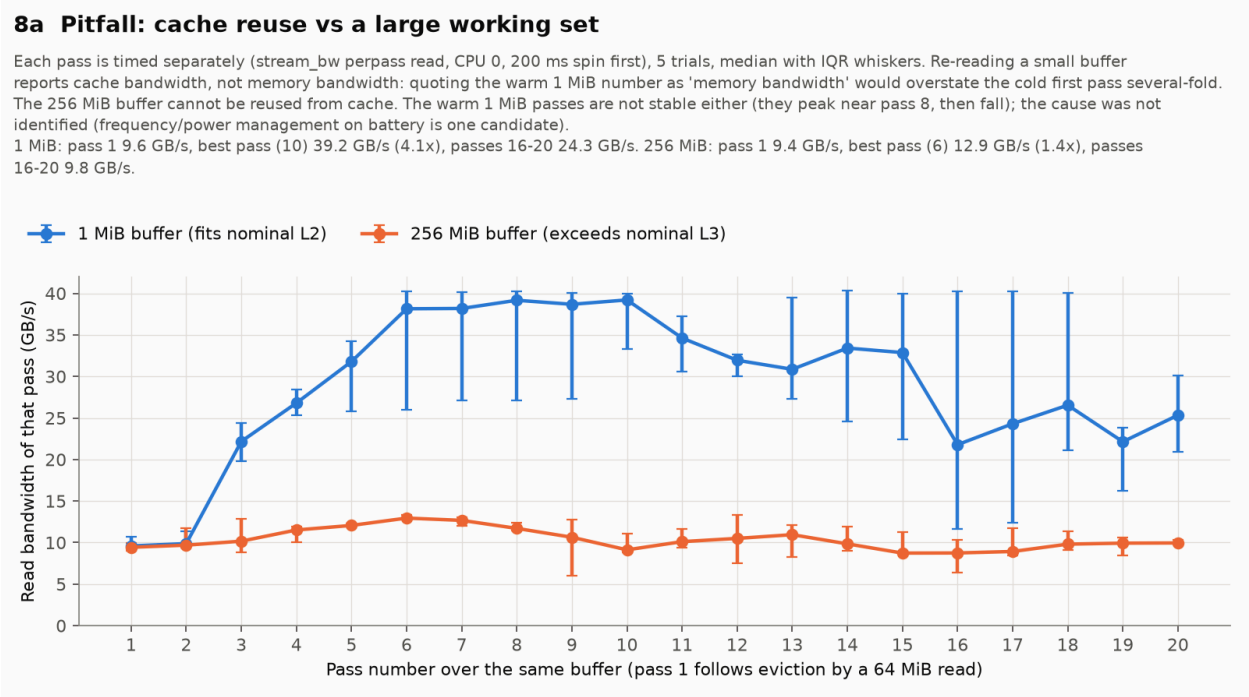


Figure O. Read bandwidth of each pass over a 1 MiB and a 256 MiB buffer after eviction.

Pinned vs Unpinned Execution

Figure P runs the same pointer chase twelve times pinned to a P-core (CPU 0), pinned to a low-power E-core (CPU 12), and unpinned. At 1 MiB the P-core takes 12.8 ns with a 13% coefficient of variation, while the E-core takes 48.5 ns, 3.8 times slower, with outliers up to 273 ns. Unpinned runs give 16.2 ns, but 7 of 12 migrated between cores and one took 186 ns, raising the coefficient of variation to 155%.

At 8 MiB the E-core is always near DRAM latency (355 ns, 1.8% variation), consistent with its lack of an L3, while the P-core is bimodal between 107 and 342 ns. None of the unpinned runs ended on an E-core here, so the original Figure A anomaly can't be blamed on E-cores. Pinning got rid of both the migration outliers and the core-type effect.

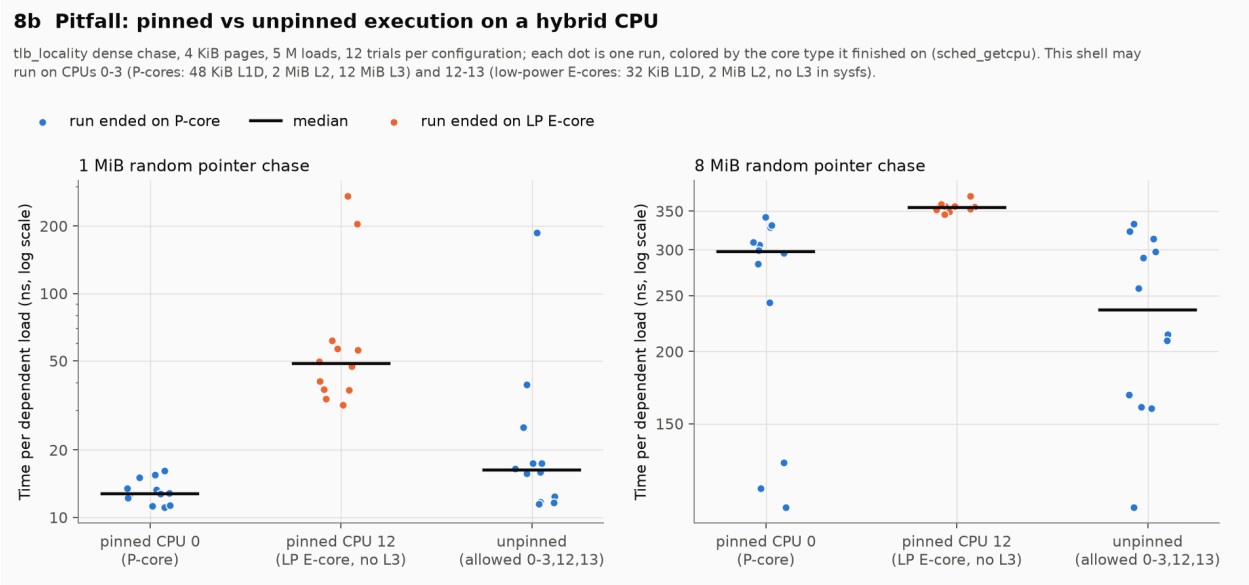


Figure P. Per-run latency pinned to a P-core, pinned to a low-power E-core, and unpinned, coloured by the core each run finished on.

Conclusion

On this laptop, dependent-load latency goes from about 3.5 ns in L1 to about 12 ns in L2, 30 ns just above L2 and 400 ns in DRAM. The L1 and L2 transitions show up near their nominal sizes, but the jump to DRAM-like latency happens between 4 and 8 MiB instead of at the 12 MiB L3.

Independent requests hide a lot of that latency. Random updates cost 30 to 42 ns instead of 400 ns, and the concurrency sweep scaled almost linearly up to 32 outstanding loads per thread. It only hit a knee with four threads at around 96 requests in flight, where more concurrency added queueing delay instead of throughput. Writes were cheaper than reads whenever the core didn't have to wait on them, and huge pages cut 10 to 15% off DRAM-resident latency by reducing translation cost.

The biggest limitations were the missing hardware counters, which meant the cache-miss and TLB sections had to rely on timing; running on battery, which kept the CPU near 2.9 GHz; huge-page backing that wasn't guaranteed; and the short runs behind the only knee I found. The pitfall experiments showed that cache reuse and unpinned execution on a hybrid CPU can each throw off a reported number by about four times.

Honest AI Usage

Claude was used to help me format this report and as a supplementary tool during this project to assist with writing plotting code for CSV datasets and to provide general coding assistance when needed. This included help with processing collected CSV data and generating visualizations for analyzing the experimental results. The AI assistance was used to support the implementation and presentation of the experiments, while the project's measurements and resulting performance analysis were based on the collected data. AI-generated code and suggestions were reviewed and adapted as needed for the project.